Decision Orchestration Platform

A source-agnostic, `n8n`-orchestrated platform for scalable, secure, resilient, AI-enabled decision automation.

This repository is a reference build of an enterprise decisioning platform — the kind of standard an automation/platform team would adopt so that every business function stops reinventing "gather evidence, get an AI opinion, have it checked, apply policy, log it, act" from scratch. The core is n8n as the orchestration engine: every trigger, HTTP call, AI agent invocation, judge call, and database write is a node in an inspectable, versioned n8n workflow graph — not logic buried in application code that only engineers can read. Around that core sit the platform standards enterprises actually need before they'll trust automation with real decisions: API-first and event-driven integration (REST, webhooks, OAuth/JWT), a layered CI/CD gate (build, lint, test-with-coverage, artifact validation, Terraform validation, secret scanning, and an adversarial AI code reviewer), infrastructure-as-code on GCP with a scale-to-zero cost model, observability (structured logging, a Cloud Monitoring dashboard, alert policies), and governance (an auditable decision ledger, least-privilege IAM, policy guardrails that gate every AI-produced recommendation before it can take effect).

The AI layer is deliberately **local-first but API-first**: a local LLM agent (Qwen, via Ollama) does the reasoning and judging by default, which keeps the platform cheap to run and keeps sensitive evidence off third-party infrastructure. But the agent talks to any OpenAI-compatible `/v1/chat/completions` endpoint (`packages/core/src/ollama.ts:OllamaClient`), so swapping in a hosted frontier model for a higher-stakes decision domain is one credential and one config value away — not a rewrite. That's the responsible-AI story in practice: run the cheap, private model by default, escalate to a stronger one deliberately, and keep the same guardrail/judge/policy layer around either.

The **evidence sources and the F1/prediction-market domain in this build are a reference implementation, not the point.** The reusable core — `packages/core` plus `workflows/00-decision-framework.json` — has zero F1-specific logic: it takes an `Evidence[]` bundle and a subject, produces an AI assessment, has an independent AI judge score that assessment against a rubric, applies a policy gate, and returns a governed decision record. Point the same core at a different set of evidence adapters and a different domain prompt and it decides for a different business problem entirely — for example: **credit/risk underwriting** (financial evidence in, approve/decline/refer recommendation out), **insurance claims triage** (claim documents + policy data in, fast-track/investigate recommendation out), **supplier selection** (RFP responses + compliance data in, award/reject recommendation out), **content moderation** (submitted content + policy rules in, allow/escalate/remove recommendation out), or **operations routing** (incident/ticket evidence in, team/priority assignment out). Swapping domains means swapping the ingest nodes and the prompt — the orchestration graph, the judge, the guardrail, and the governance ledger stay exactly the same.

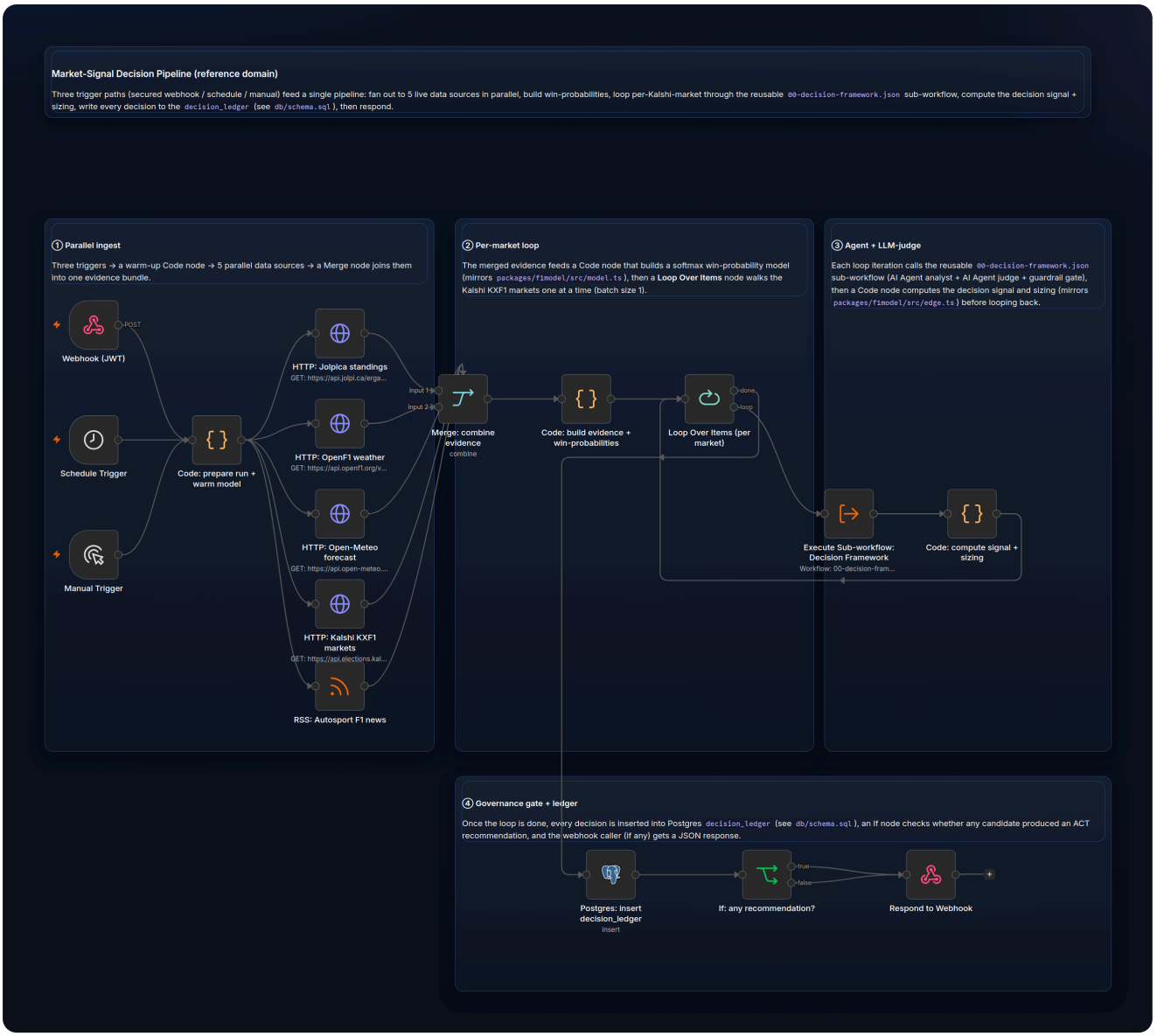
Attribute → implementation matrix

#	Attribute	How the platform delivers it — with the artifacts that prove it
2.1	**Scalable**	**Scales *out* under load, not just to zero.** The same decision graph runs unchanged whether it evaluates 10 candidates or 10,000: queue-mode spreads work across horizontally-scaled `n8n worker` containers behind a Redis/BullMQ queue, the LLM tier autoscales as a GPU MIG (0→2 L4s), and Cloud Run adds instances on concurrency — then all three fall back to zero when idle to control cost. (`infra/terraform/{queue_mode,ollama_gpu,cloudrun_n8n}.tf`, gated by `enable_queue_mode` / `enable_gpu`.)
2.2	**Secure integration — OAuth/JWT + webhooks**	**Every entry point is authenticated and every secret is externalized.** The inbound webhook is JWT-verified, machine-to-machine calls to the market-signal API are RSA-PSS-signed, CI reaches GCP with no long-lived keys via Workload Identity Federation, and no credential ever lives in code or workflow JSON. (`workflows/10-f1-edge-flagship.json` · `packages/kalshi/src/signer.ts` · `infra/terraform/{wif,secrets}.tf`.)
2.3	**Resilient**	**A transient failure never loses a decision.** Every external call retries with backoff, n8n catches node-level errors, Pub/Sub dead-letters what can't be processed, and Cloud Run + the GPU MIG self-heal on liveness probes — while Postgres durably records every run so any decision can be replayed. (retry options in `workflows/*` · `infra/terraform/pubsub.tf` · `db/schema.sql`.)
2.4	**AI-enabled**	**Reasoning is a first-class, swappable step.** A local LLM agent turns raw evidence into a structured, defensible assessment, and an independent LLM-as-judge scores that reasoning before it can move a decision — all over an OpenAI-compatible interface, so trading the local model for a hosted frontier model is one config value, not a rewrite. (`workflows/00-decision-framework.json` · `packages/core/src/{agent,judge}.ts`.)
2.5	**Multi-function decisioning framework**	**One decision engine, any domain.** The core ingest → agent → judge → guardrail → record pipeline carries zero F1-specific logic; the F1 build is just one caller. Point it at fraud signals, credit files, or claims triage and the same governance applies. (`packages/core` · `workflows/00-decision-framework.json` — see Introduction.)
2.6	**API-first / REST**	**Everything is reachable over REST.** n8n exposes its own public REST API, and the flagship returns each governed decision as JSON to any caller through a Respond-to-Webhook node — so an existing system can request a decision and consume the result without touching the internals. (`workflows/10-f1-edge-flagship.json`.)
2.7	**Event-driven**	**Decisions fire on events, not manual runs.** The pipeline triggers from an inbound webhook, a schedule, or an API/manual call; warm-up runs on a timer; and every decision is published to a Pub/Sub topic other systems can subscribe to. (three triggers in `workflows/*` · `infra/terraform/pubsub.tf`.)
2.8	**Connects enterprise systems, automation platforms, and AI services**	**Eight heterogeneous systems, one graph.** A market-signal API, championship data, two weather services, a news feed, the local LLM runtime, Postgres, and Pub/Sub are wired together in a single workflow — each behind a typed adapter, so a source can be swapped without touching the pipeline. (see Data sources.)
2.9.1	**Development standard**	**Quality is enforced by machines, not good intentions.** TDD with a hard coverage gate blocks undertested merges, and a *second* AI — a local Qwen adversarial reviewer — critiques every PR's diff in CI; workflow JSON is validated against n8n's own node schemas. Compatible with the **n8n-mcp** toolchain and **n8n-skills** for AI-assisted

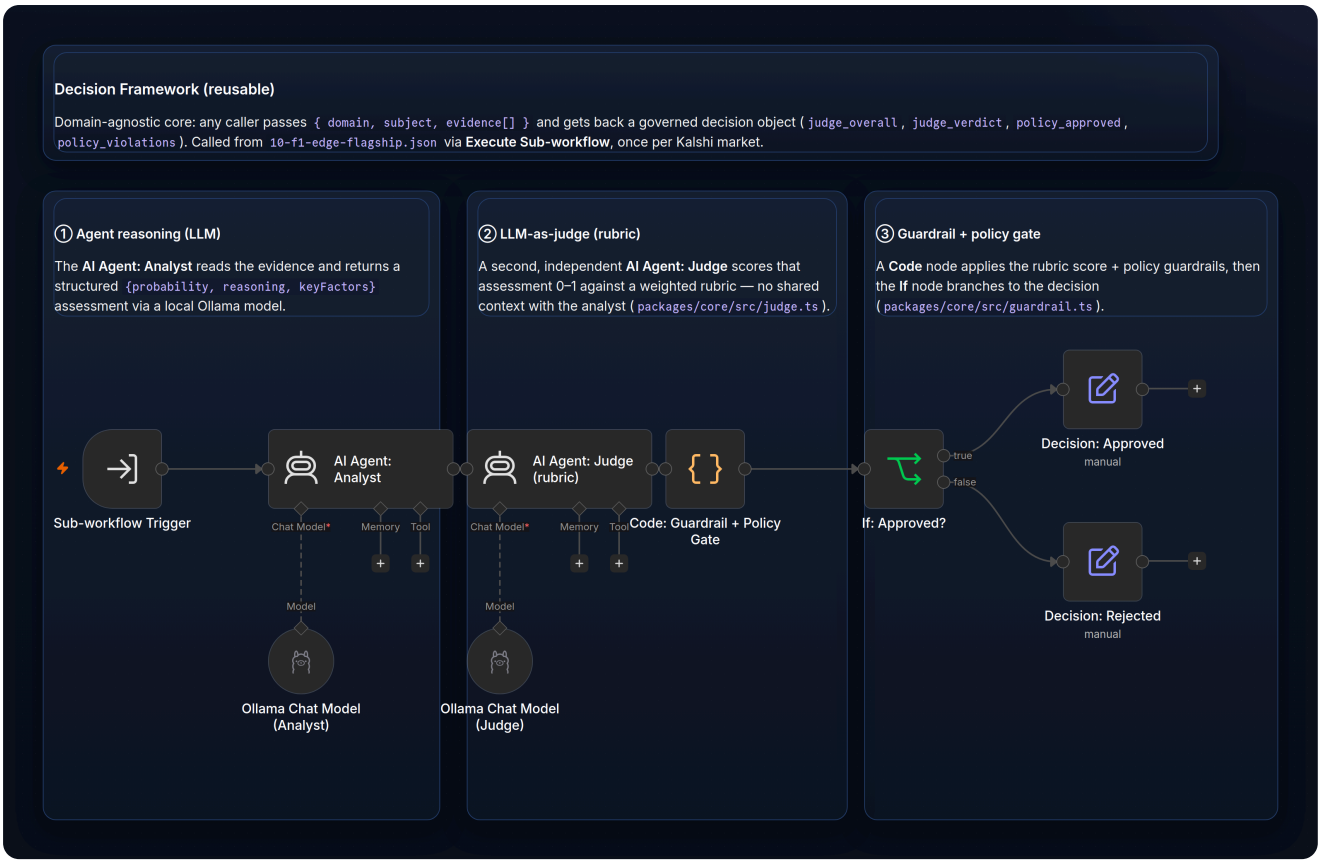
#	Attribute	How the platform delivers it — with the artifacts that prove it
		authoring. (`vitest.config.ts` · `scripts/ci/ai-review.mjs` · `.github/workflows/ci.yml`.)
2.9.2 / 2.9.6	**Deployment / environment strategy**	**Ship the whole platform from a clean checkout, repeatably.** Infrastructure is declarative Terraform, releases deploy through GitHub Actions over keyless WIF, and cost/topology are dialed per environment with documented toggles (`enable_gpu`, `enable_queue_mode`, `n8n_public`). (`infra/terraform/` · `.github/workflows/cd.yml`.)
2.9.3 / 2.9.8	**Monitoring / observability**	**You can see what the platform is deciding *and* whether it's healthy.** A provisioned **Grafana** dashboard reads the `decision_ledger` (volume, approval rate, judge scores, action mix) for decision analytics; Cloud Monitoring adds an infra dashboard, a log-based metric on decision events, and a 5xx alert policy — all as code. (`infra/grafana/` · `infra/terraform/monitoring.tf` + `dashboards/n8n-overview.json` — see Observability.)
2.9.4	**Reliability**	**State survives restarts and is auditable after the fact.** Cloud SQL Postgres durably holds both n8n's own state and the decision ledger, with full provenance (`decision_runs` / `evidence_snapshots` / `decision_ledger`) so any run can be reconstructed. (`infra/terraform/cloudsql.tf` · `db/schema.sql`.)
2.9.5	**Governance**	**No decision escapes policy or the audit trail.** A deterministic guardrail/policy gate runs independently of the AI, every decision is written to an immutable ledger with its rationale and any policy violations, and access is least-privilege IAM with all secrets externalized. (`packages/core/src/guardrail.ts` · `db/schema.sql` · `infra/terraform/{iam,wif}.tf`.)
2.9.7	**Reusable frameworks / templates / prompt patterns**	**The reusable parts are packaged for reuse.** The domain-agnostic sub-workflow, the parameterized agent/judge prompt builders, and the typed contracts are all callable building blocks with no domain strings baked in. (`workflows/00-decision-framework.json` · `packages/core/src/{agent,judge}.ts:buildPrompt`.)
2.9.9	**Cloud (GCP)**	**Cloud-native and containerized end to end.** n8n runs as a container on Cloud Run, the LLM tier on a GCP GPU MIG, and the *identical* stack runs locally via Docker Compose — no "works on my machine" gap between demo and production. (`docker-compose.yml` · `infra/terraform/{cloudrun_n8n,ollama_gpu}.tf`.)

The n8n workflow graph

The workflow graph *is* the platform's logic surface — triggers, HTTP calls, the AI agent, the judge, and the database write all run as nodes on a canvas that a non-engineer can open, read, and reason about. The images below are **screenshots of n8n's own canvas** — the actual workflow JSON in `workflows/` imported into a running n8n instance and rendered by n8n itself (`make import-workflows`), not a mock-up: standard n8n nodes with their real icons, connectors, and sticky-note annotations.



> `00-decision-framework` **— the domain-agnostic decision core**, highlighted because it's the reusable part of this platform. 9 executable nodes: agent → judge → guardrail/policy gate → decision. It has no F1-specific logic at all — swap the evidence sources and the domain prompt feeding it, and it decides for any domain (see the example domains in the Introduction).



Both images are regenerated by `scripts/render-n8n-live.mjs` (`make graph`), which imports each workflow into a real n8n instance and screenshots the canvas — so the diagram can never drift from the JSON that actually runs. A third workflow, `workflows/90-warmup-ping.json` (6 nodes), polls Ollama and pre-warms the model on a schedule so the first Agent/Judge call of a session doesn't eat a multi-minute cold-load penalty — see `workflows/README.md` for the full annotated walkthrough of every node.

The decision pipeline, explained

Every decision — regardless of domain — moves through the same seven stages, implemented once in `packages/core` and mirrored 1:1 as n8n nodes in `workflows/00-decision-framework.json` + `workflows/10-f1-edge-flagship.json`. The business point of this structure: **the policy thresholds, the judge's rubric, the guardrail rules, and the workflow graph itself are all data and configuration, not compiled application code.** An analyst or risk owner can open the n8n canvas, change a threshold in `.env`, or edit the rubric weights in `packages/contracts/src/rubric.ts` without a developer touching a compiler — the logic that governs a decision is owned by the business, not hidden inside a binary.

Stage	Business purpose	How it's implemented — flagship (F1) → framework (domain-agnostic)
1 · Ingest	Pull every piece of relevant evidence from its system of record before forming an opinion	**Flagship:** 5 parallel HTTP/RSS calls (championship data, weather ×2, market-signal API, news), retried and merged into one bundle (`packages/app/src/evidence.ts`). **Framework:** a typed, Zod-validated `Evidence[]` contract (`packages/contracts/src/schemas.ts`).
2 · Enrich	Turn raw evidence into a comparable, quantitative	**Flagship:** softmax win-probability model + model-vs-market signal (`packages/f1model/src/{model,edge}.ts`). **Framework:** domain-

Stage	Business purpose	How it's implemented — flagship (F1) → framework (domain-agnostic)
	estimate	supplied `context` on `DecisionInput`.
3 · AI agent	Produce a structured, reasoned assessment instead of an opaque score	**Flagship:** local Qwen emits `{probability, reasoning, keyFactors}` from evidence only (`packages/core/src/agent.ts`). **Framework:** n8n **AI Agent: Analyst** + **Ollama Chat Model**.
4 · LLM-as-judge *(responsible-AI gate)*	Independently check the AI's reasoning quality before it can move a real decision	**Flagship:** a second, independent Qwen scores the assessment 0–1 against a human-authored weighted rubric (`packages/core/src/judge.ts`, `packages/contracts/src/rubric.ts`). **Framework:** n8n **AI Agent: Judge**.
5 · Guardrail / policy gate	Enforce business policy deterministically, independent of what the AI concluded	**Flagship:** pre-assessment fast-fail + post-assessment policy check (`packages/core/src/guardrail.ts`). **Framework:** n8n **Code: Guardrail + Policy Gate**.
6 · Decision record	Produce an immutable, schema-checked artifact that can be replayed or audited later	**Flagship:** Zod-validated immutable record (`packages/core/src/decision.ts` + `DecisionRecordSchema`). **Framework:** n8n **If: Approved?** → **Decision** nodes.
7 · Action	Turn a governed decision into a recommendation a downstream system or human can act on	**Flagship:** act / hold / decline + half-Kelly sizing, written to the `decision_ledger` table and returned via the webhook (`packages/app/src/persist.ts`, `db/schema.sql`). **Framework:** returns the governed decision; the caller defines what "action" means.

`packages/core/src/decision.ts:runDecision` is the orchestrator: input-policy check → agent assess → signal computation → judge → (revise loop, up to `maxRevise`) → decision-policy check → record. The **adversarial LLM-as-judge** is worth calling out specifically: it doesn't just rubber-stamp the agent's output — it's instructed to score the agent's reasoning critically against the rubric, and a low score sends the assessment back for revision or has it rejected outright before it ever reaches the guardrail or the ledger. That's what makes it safe to run a smaller, cheaper local model as the primary agent: a bad or ungrounded assessment is caught and stopped, not silently passed through as a governed recommendation. The whole pipeline is deterministic and unit-testable via `StubLLM` (`packages/core/src/ollama.ts`) with zero network/GPU dependency — that's how 141 tests run in under a second (see Testing & Quality Harness).

A real decision run

Below is real, reproducible output from `node packages/app/dist/cli.js --mode fixture --stub --limit 9`, run against captured fixture data in this repo (deterministic `StubLLM`, no GPU needed). This is an excerpt — three of the nine candidates evaluated in that run — chosen because it shows both outcomes side by side (run it yourself for the full nine-row output):

```
SUBJECT                                              MODEL  MARKET  SIGNAL  ACTION  JUDGE  STAT
Will Oscar Piastri win the F1 Drivers Championship?  0.183  0.235   -0.052  PASS    pass   appr
```

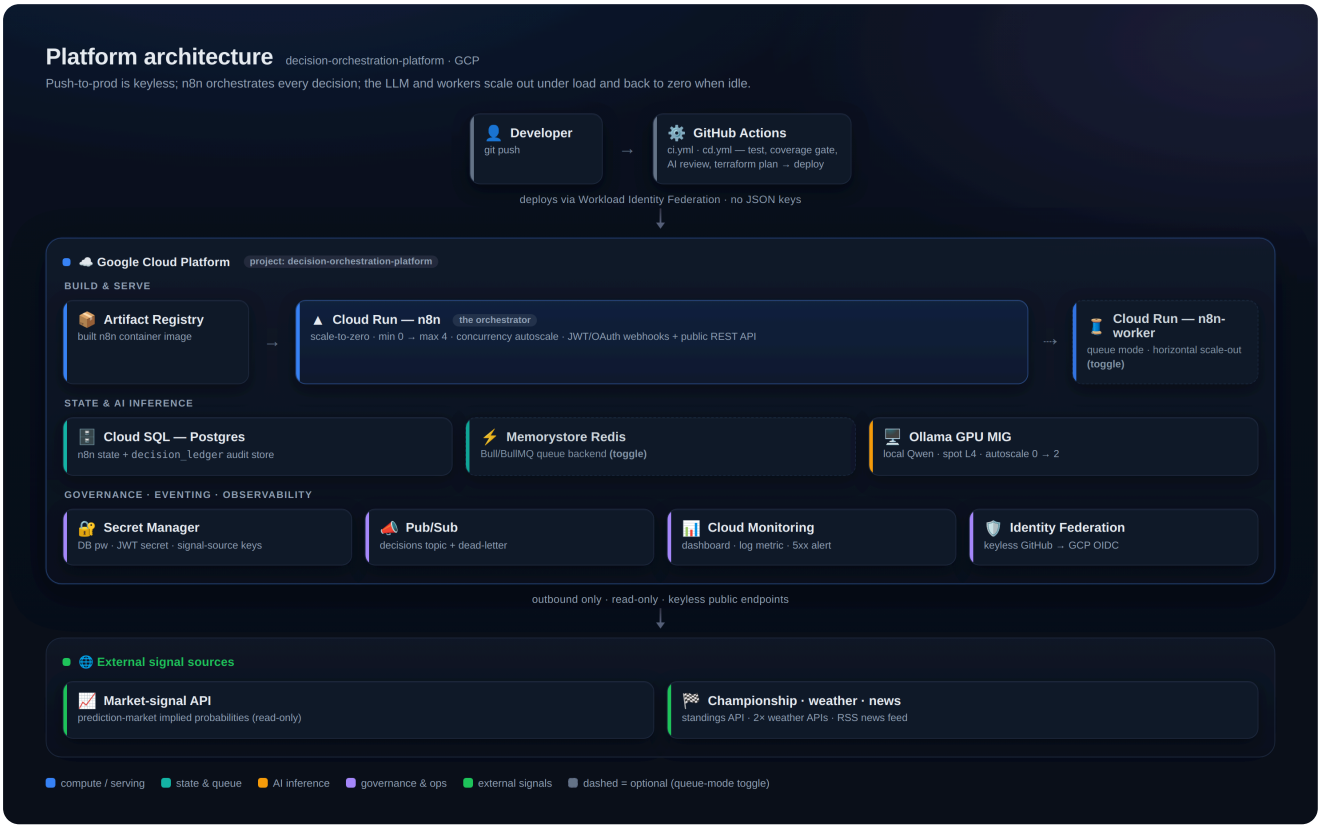
```
Will Max Verstappen win the F1 Drivers Championship?   0.121  0.315   -0.194  PASS    pass   appr
Will Lando Norris win the F1 Drivers Championship?     0.200  0.135   +0.065  BUY     pass   appr
```

Read this as a business decision example: for each candidate, the system's model estimates a probability (`MODEL`) and compares it against an external market-implied probability (`MARKET`) — the gap between the two is the decision **signal**. For Norris, the softmax model (`packages/f1model/src/model.ts`) estimates a **0.200** win probability, but the external market-signal source implies only **0.135** — a **+0.065 signal** that clears the policy's `minEdge` (0.05) and `minConfidence` (0.55) thresholds (`packages/f1model/src/edge.ts:decideAction`), so the governed recommendation is **act** (`BUY` in the raw output — see Terminology note below). Piastri's and Verstappen's signals are negative — the market already prices them higher than the model does — so the system correctly recommends **decline** (`PASS`). In every row, an independent adversarial judge scored the reasoning `pass`, the policy gate approved it, and the outcome was written to the auditable `decision_ledger` as `status: approved`.

A note on the action codes

The pipeline's code and raw CLI output use the short codes `BUY` / `HOLD` / `PASS` for historical reasons tied to the reference domain — read them as decision recommendations, not trading instructions: `BUY` means **recommend act**, `HOLD` means **hold — insufficient signal or confidence to act**, and `PASS` means **decline — signal does not clear policy**. The `decision_ledger` table these are written to is an auditable decision record for governance and traceability, not a trade log — there is no order-execution code path anywhere in this repository (`packages/kalshi/src/client.ts` is read-only by design; see Design decisions).

Architecture



Deployed live to a GCP project (`infra/terraform/`) provisioned for this platform. n8n runs on Cloud Run and the Ollama GPU MIG is `terraform apply`-ed (instance template, autoscaler, internal load balancer; n8n auto-wired

to it) — it idles at zero and warms on demand (see Deploy to GCP and `BLOCKERS.md`).

Testing & Quality Harness

This is the most heavily invested-in part of the repo — TDD throughout, with tests at every seam so the AI-in-the-loop pieces stay honest.

Local: 141 tests, 17 files

```
Test Files  17 passed (17)
     Tests  141 passed (141)
  Duration  872ms
```

Package	Stmts	Branch	Funcs	Notes
`@dop/contracts`	100%	98.6%	100%	Every external payload (championship data, market signal, weather, news) validated by a Zod schema, plus the judge rubric and policy loader.
`@dop/kalshi`	100%	93.6%	95.8%	Read-only external market-signal client. Includes an RSA-PSS **signature round-trip test** (`signer.test.ts`) verifying `signKalshiRequest` against Node's own `crypto.verify`.
`@dop/f1model`	100%	99.0%	100%	Softmax model + signal/sizing math — pure functions, exhaustively tested.
`@dop/core`	99.6%	94.9%	96.8%	The reusable decisioning framework: agent, judge, guardrail, decision assembly, `StubLLM`. Deterministic pipeline tests need **no GPU, no network**.
`@dop/app`	74.2%*	87.9%	92%	End-to-end pipeline + evidence gathering are fully covered; `cli.ts` (a thin argv-parsing entrypoint) and `persist.ts`'s Postgres-backed `PgDecisionStore` (exercised at runtime against a real DB, not mocked in unit tests) pull the package average down.

* The monorepo-wide aggregate across all 5 packages is **93.56% statements / 94.52% branches / 95.83% functions** (v8 coverage, `npm run test:cov`) — every runtime library package clears **100% statements**; only the CLI entrypoint and the live-DB persistence adapter (intentionally not unit-mocked — see `packages/app/src/persist.ts`'s own doc comment) bring the package-level average for `@dop/app` down. All of this clears CI's enforced floor of **80% lines/functions/statements, 75% branches** (`vitest.config.ts`) with real margin.

Run it yourself:

```
npm run test:cov     # vitest run --coverage — fails the process if under threshold
make test-cov        # same, via the Makefile
```

CI: 6 layered gates on every PR (`.github/workflows/ci.yml`)

Gate	What it checks	Blocking
`build-test`	`tsc -b` → `eslint` → `vitest run --coverage` (80/80/75/80 threshold, enforced by `vitest.config.ts`)	Yes
`validate-artifacts`	Every `fixtures/**/*.json` parses; every `workflows/*.json` is a structurally valid n8n export (`nodes[].type`, `connections`) — `scripts/ci/validate-artifacts.mjs`	Yes
`terraform`	`terraform fmt -check` / `init -backend=false` / `validate` on `infra/terraform/`	Yes
`security`	`gitleaks` secret scan (blocking) + `npm audit --audit-level=high` (advisory)	Secret scan only
`adversarial-review`	See below	Yes, on high-severity findings

The adversarial reviewer is the headline governance gate. It installs Ollama *inside the GitHub Actions runner*, pulls `qwen2.5:3b`, computes the PR's diff, and sends it to a system prompt (`scripts/ci/ai-review.mjs`) explicitly instructed to be adversarial: *"assume the author missed something and actively try to prove the diff is unsafe, incorrect, undertested, or overcomplicated."* It scores against a fixed rubric (correctness, security, test coverage, error handling, simplicity), parses the model's JSON findings, posts them as a PR comment / step summary, and **blocks the merge on any `high`-severity finding** — unless the PR carries the `override-ai-review` label (with an explanation left in a PR comment). This same responsible-AI pattern — a governed, rubric-scored AI check with a human override path — is the pattern the decision pipeline itself uses for the LLM-as-judge stage. No hosted AI API is required anywhere in this pipeline — the whole review runs on a 3B model inside the free GitHub Actions runner. Infra flakiness (Ollama fails to start, model pull fails, output doesn't parse) degrades gracefully to a non-blocking "skipped" notice rather than failing CI on infrastructure, not code.

The workflows are version-controlled JSON, validated against n8n's own node schemas by importing into a live n8n instance in CI. That schema-first surface is what makes the platform work hand-in-glove with **n8n-mcp** (programmatic node discovery and workflow-JSON validation against a live instance) and **n8n-skills** (repeatable, AI-assisted workflow-authoring patterns) for ongoing development.

Quickstart

Runs **clone → up with zero credentials**, against real captured fixture data (see Data sources):

```
git clone <this-repo> && cd n8n
cp .env.example .env        # optional — sane defaults are baked in
docker compose up -d        # postgres + n8n + ollama
make warm                   # pull qwen2.5:7b-instruct / qwen2.5:14b-instruct into ollama
make import-workflows       # import workflows/*.json into the running n8n
make e2e                    # run the end-to-end decision pipeline script
```

Or skip Docker/n8n entirely and run the TypeScript pipeline directly:

```
npm install && npm run build
node packages/app/dist/cli.js --mode fixture --stub --limit 9   # deterministic, no GPU
```

```
node packages/app/dist/cli.js --mode fixture --model qwen2.5:14b-instruct  # real local LLM
```

`make help` lists every lifecycle target (`up` / `down` / `logs` / `seed` / `graph` / `pdf` / `bootstrap-gcp` / `deploy` / `gpu-up` / `gpu-down` /...).

Deploy to GCP

```
export PROJECT_ID=<your-project-id> REGION=us-central1
make bootstrap-gcp   # one-time: create project, link billing, enable APIs, create tfstate bucket
make deploy          # terraform init + apply (infra/terraform/)
```

- **Scale-to-zero by default**: `min_instances=0` on the n8n Cloud Run service (`infra/terraform/variables.tf`) — idle cost is near-zero.
- **GPU inference tier scales to zero**: `enable_gpu=true` (the default) provisions the spot **L4** Ollama MIG behind an internal load balancer, autoscaling **0→2** and sitting at **zero when idle** — so it costs nothing until a run warms it. L4 quota is granted on the deployed project, so `make deploy` brings the tier up; `make gpu-up` (or the warm-up workflow) scales it to 1 before a demo and `make gpu-down` returns it to zero. Set `enable_gpu=false` for a CPU-only demo.
- **CD is opt-in**: `.github/workflows/cd.yml` deploys on push to `main` via Workload Identity Federation (no long-lived GCP keys), but only runs when the `ENABLE_CD` repo variable is `true` — a deliberate safety gate documented in `.github/workflows/README.md`.

Running it

Two ways to watch a governed decision get produced end to end.

Locally — zero credentials, no GPU required (~2 min):

```
docker compose up -d          # postgres + n8n + ollama
make warm                     # pull the Qwen models into local Ollama
make import-workflows         # load workflows/*.json into n8n
```

Then either run it headless from the CLI —

```
make e2e                      # prints the SUBJECT / MODEL / MARKET / SIGNAL / ACTION / JUDGE / S
```

— or open **http://localhost:5678**, open **Market-Signal Decision Pipeline**, and click **Execute workflow**. The CLI path is fully deterministic (`--stub`), so it needs no GPU at all.

On the live GCP instance:

```
# 1. sync the version-controlled workflows into the live n8n over its API (idempotent).
#    Manual run — authenticate with your n8n owner login:
N8N_BASE="$(terraform -chdir=infra/terraform output -raw n8n_url)" \
  N8N_EMAIL=you@example.com N8N_PASSWORD='<your n8n password>' make import-workflows-remote
#    (CD does this automatically on push using an n8n API key from Secret Manager
```

```
#     instead of a password — see cd.yml; you don't need a key for a manual run.)
# 2. warm the GPU inference tier (spot L4; first run pulls the model, ~3-4 min)
make gpu-up   PROJECT_ID=f1-decision-platform REGION=us-central1
# 3. open the live n8n, open the flagship workflow, and click Execute
#    — or POST the JWT-secured webhook on the Webhook node
# 4. release the GPU when done
make gpu-down PROJECT_ID=f1-decision-platform REGION=us-central1
```

Workflows are **version-controlled and deployed programmatically** — `workflows/*.json` in git is the source of truth, imported via the n8n API by `make import-workflows-remote` (and by `cd.yml` on every push). The credential *values* (Ollama URL, Postgres, JWT) are set once per instance via the UI/API, since n8n deliberately never exports secrets in workflow JSON. Every run writes a row to the `decision_ledger` table and returns the decision as JSON.

What it costs to run. Because the model is **local** (no per-token API fee), cost is GPU wall-clock, not per call:

Scenario	Cost
Idle — everything scaled to zero	**~$8/mo** — just the Cloud SQL `db-f1-micro`
Spot L4 GPU while warm	**~$0.25/hr** (≈ $0.004/min)
One full 9-candidate decision run, warm	~2 min GPU ≈ **$0.01**
First run of a session (cold model load)	**+~$0.02** one-time (~3–4 min)
~100 full demo runs	**≈ $1** of GPU + the flat ~$8/mo DB

Cloud Run n8n is scale-to-zero (pennies per run); the GPU bills only while scaled up, so a 10-minute demo is a few cents. `make destroy` tears everything down.

Observability

Two layers: a **decision-analytics dashboard** you run locally, and **infra health + alerting** in GCP.

Local — Grafana over the decision ledger. `docker compose up` also starts **Grafana** (http://localhost:3000), auto-provisioned with a Postgres datasource and a dashboard on the `decision_ledger` table — zero manual setup. It surfaces decision volume, policy-approval rate, the LLM-judge pass rate and average score, the action mix, and the full auditable row-level ledger:

Everything is version-controlled in `infra/grafana/` (`provisioning/datasources/`, `provisioning/dashboards/`, `dashboards/decisions.json`) and provisioned on boot — the dashboard is code, not a click-ops artifact. (If port 3000 is taken, set `GRAFANA_PORT`.)

GCP-native — Cloud Monitoring + alerting. In the deployed stack, health monitoring is Google-native and ships as Terraform: a Cloud Monitoring **dashboard** (`infra/terraform/dashboards/n8n-overview.json`), a **log-based metric** counting decision events (`google_logging_metric.decisions_logged`), and a **5xx-rate alert policy** on the n8n Cloud Run service (`google_monitoring_alert_policy.n8n_error_rate`) — all in `infra/terraform/monitoring.tf`, with structured logs from every run flowing to Cloud Logging.

Together they answer two different questions: the Grafana board shows **what the platform is deciding and whether the AI is being governed well**; Cloud Monitoring shows **whether the platform is healthy**.

Project layout

```
packages/
  contracts/   @dop/contracts — shared types, Zod ingestion schemas, judge rubric, policy loader
  kalshi/      @dop/kalshi    — read-only external market-signal client, RSA-PSS signer, fixture-
  f1model/     @dop/f1model   — softmax probability model, model-vs-market signal calc, half-Kell
  core/        @dop/core      — the reusable decision framework: agent, judge, guardrail, decisio
  app/         @dop/app       — end-to-end pipeline + the `f1-decision` CLI
fixtures/      real captured snapshots of every external data source (see below)
workflows/     importable n8n workflow JSON exports (the visual centerpiece)
db/            decision_runs / evidence_snapshots / decision_ledger schema
infra/terraform/  GCP IaC — Cloud Run, Cloud SQL, Ollama GPU MIG, Pub/Sub, Secret Manager, WIF, m
```

```
scripts/      bootstrap-gcp.sh, deploy.sh, export-graph.mjs, ci/ai-review.mjs, ci/validate-artif
docs/img/     architecture figure + PNG screenshots of every workflow rendered from live n8n
```

Data sources

Every evidence source is a **pluggable adapter**, not a hardcoded integration: each one is an n8n `HTTP Request` or `RSS Read` node paired with a typed `@dop/*` client and a Zod schema that validates the payload on the way in (`packages/contracts/src/schemas.ts`). Swapping a data source for a different domain means pointing the node at a new endpoint and writing a new Zod schema for its response shape — nothing else in the pipeline changes. This is the source-agnostic story in practice: the platform doesn't know or care that today's evidence happens to be F1 championship data. Secure integration is handled the same way regardless of source — OAuth2/JWT credential types in n8n, secrets in GCP Secret Manager, and RSA-signed requests where the upstream API requires it (`packages/kalshi/src/signer.ts`).

Source	Signal it provides	Swap-in note
Jolpica-F1	Championship standings, race results (Ergast-compatible)	Any REST API returning structured domain state — replace with a claims system, a CRM, or an ERP feed
OpenF1	Session metadata, trackside weather telemetry	Any real-time telemetry/event API
Open-Meteo	Race-weekend weather forecast	Any contextual/environmental data API
Autosport	F1 news RSS feed	Any RSS/news feed — swap for an industry news feed or an internal announcements feed
Kalshi	External market-implied probability (the model-vs-market decision signal)	Any external benchmark/reference-rate API — a credit bureau score, an actuarial table, a market index

`fixtures/` holds real, captured snapshots of every one of the above (see `fixtures/README.md` + `fixtures/manifest.json`), so `docker compose up` runs **clone → run with zero credentials**. Note: the market-signal source's real F1-2026 markets were captured with a genuinely empty off-season order book (`yes_bid` / `yes_ask` null); those 22 markets were synthetically demo-priced (flagged `_demo_priced: true`, real thin values preserved alongside) so the signal math has something to compute against — see `fixtures/README.md`'s "Kalshi demo pricing" section for full transparency on exactly which numbers are synthetic.

Design decisions

- **Source-agnostic framework, pluggable reference domain.** `packages/core` and `workflows/00-decision-framework.json` contain zero domain-specific logic — the F1/prediction-market build in this repo is one instantiation, chosen because it has genuinely live, free, keyless APIs to demonstrate the full pipeline end to end. Swapping the evidence adapters and the domain prompt (`packages/core/src/agent.ts:buildPrompt`) retargets the same core to a different business decision.
- **Local-LLM-first, API-first by design.** `packages/core/src/ollama.ts:OllamaClient` talks to any OpenAI-compatible `/v1/chat/completions` endpoint. The shipped configuration (`.env.example`, `docker-compose.yml`, the n8n workflow's `lmChatOllama` nodes) runs entirely on a local Ollama model —

auditable, cost-free to run, reproducible offline, and keeps evidence off third-party infrastructure. Pointing the same interface at a hosted frontier model for a higher-stakes decision domain is a config change, not an architecture change.

- **Humans own the logic.** The policy thresholds (`.env`, `packages/core/src/guardrail.ts`), the judge's rubric (`packages/contracts/src/rubric.ts`), and the entire workflow graph (`workflows/*.json`, viewable on the n8n canvas) are inspectable, versioned configuration — a policy owner or analyst can change what the platform will and won't approve without a developer touching compiled code.
- **Fixture-backed, zero-credential onboarding.** `KALSHI_MODE=fixture` and the equivalents for every other source default to on everywhere (`.env.example`, `packages/app/src/evidence.ts`), backed by real captured API snapshots in `fixtures/`. Anyone can clone and run the full pipeline with no API keys; flipping to live mode is a config change per source, and only order-*placement*-style write paths (which this platform doesn't implement — see below) would ever need signing credentials.
- **Read-only by construction on the reference domain.** There is no order-submission code path anywhere in `@dop/kalshi` (see the file-level comment in `packages/kalshi/src/client.ts`: *"Paper-only: order execution intentionally not implemented; this client is read-only"*). Every governed recommendation is a logged, auditable entry in `decision_ledger` — a decision record, not a transaction.
- **Deterministic-by-construction library code.** Only `packages/app/src/cli.ts` is allowed to touch `Date.now()` / `randomUUID()` — every other module takes `createdAt` / `id` as parameters, which is what makes the 141-test suite fast and hermetic (see the doc comment at the top of `cli.ts`).
- **GPU inference scales to zero.** The Ollama tier runs on a spot-L4 GPU MIG that autoscales **0→2** and idles at zero — the model is warmed on demand (`make gpu-up` or the warm-up workflow) before a run and released after, so the only always-on GCP cost is the small Cloud SQL instance (~$8/mo). n8n is live on Cloud Run; the identical local Ollama stack runs the full pipeline offline with no cloud at all.